

Technical Report: Cloud Computing & Containerization

Final Course Project - Task Management API

2026

Contents

1	Executive Summary	1
2	Team Contributions	2
3	Virtualization vs. Containerization Analysis	2
4	System Architecture	2
5	Network Architecture	3
6	Storage Architecture & Persistence	3
6.1	Persistence Verification	3
7	Dockerfile Layer Optimization Analysis	3
8	Security Practices	4
9	Failure-Mode Scenarios	4
10	Testing Evidence	4
11	Registry & Versioning	4
12	Conclusion	5

1 Executive Summary

This report documents the architecture, implementation, and verification of the Task Management API, a containerized Cloud Computing final project. The

project demonstrates a modern microservices-oriented approach using Node.js for the API, PostgreSQL for relational data storage, and Nginx as a reverse proxy. By leveraging Docker and Docker Compose, the system achieves consistent deployment, high isolation, and reliable persistence. All components have been securely configured, tested for failure modes, and versioned on Docker Hub.

2 Team Contributions

Team member names/contribution allocation require completion before final submission.

- [Team Member 1 Name]: Backend Development & Containerization
- [Team Member 2 Name]: Reverse Proxy Configuration & Security
- [Team Member 3 Name]: Database Architecture & Storage
- [Team Member 4 Name]: Testing & Failure Mode Verification

3 Virtualization vs. Containerization Analysis

In traditional virtualization, deploying this 3-tier architecture would require three separate Virtual Machines (VMs), each running a full Guest Operating System. This results in heavy CPU/RAM overhead, slow startup times, and wasted disk space.

By utilizing **Containerization (Docker)**: - **Shared Kernel**: All containers share the host machine's OS kernel, eliminating the need for Guest OS layers. - **Lightweight Footprint**: We utilized Alpine Linux base images (ode:24-alpine and postgres:15-alpine), reducing the image size drastically (often under 100MB). - **Near-Instant Startup**: Containers spin up in milliseconds compared to the minutes required for VMs. - **Portability**: The declarative compose.yaml guarantees that the environment runs identically on any developer's machine.

4 System Architecture

The system follows a 3-tier architecture:

Client -> Nginx (:8080) -> Node.js/Express API (:3000) -> PostgreSQL (:5432)

- **Nginx (Reverse Proxy)**: Acts as the sole entry point. It receives HTTP requests on port 8080 and proxies them to the internal API on port 80.
- **Node.js API**: Handles the business logic and REST CRUD operations.
- **PostgreSQL**: Stores the tasks relationally.

5 Network Architecture

The project employs a custom Docker bridge network named ask-management-network. - **Isolation:** The ask-management-api and ask-management-db containers do NOT publish any ports to the host machine. They are completely isolated and can only be accessed from within the Docker network. - **DNS Resolution:** Containers communicate using Docker's internal DNS. The API connects to the database using the hostname ask-management-db. Nginx routes traffic to the API using the hostname ask-management-api.

6 Storage Architecture & Persistence

By default, data inside a container is ephemeral and lost when the container is removed. To ensure data persistence for PostgreSQL, a **Named Docker Volume** was implemented.

- **Volume Name:** ask-management-db-data
- **Mount Point:** /var/lib/postgresql/data (Inside the DB container)
- **Declarative Config:** Defined in compose.yaml under olumes: to ensure Docker manages its lifecycle independently of the containers.

6.1 Persistence Verification

To verify that data persists across container lifecycles: 1. A new task was created via POST /api/tasks. 2. The containers were gracefully stopped and removed using docker compose down. (*Note: docker compose down -v was strictly avoided, as the -v flag destroys volumes*). 3. The cluster was restarted using docker compose up -d. 4. A GET /api/tasks request confirmed that the previously created tasks remained intact.

7 Dockerfile Layer Optimization Analysis

The Dockerfile for the Node.js API was optimized for layer caching and minimal size:

```
dockerfile FROM node:24-alpine WORKDIR /usr/src/app COPY package*.json
./ RUN npm install --omit=dev COPY . . USER node EXPOSE 3000 CMD
["npm", "start"]
```

- **Layer Caching:** package.json is copied and pm install is executed *before* copying the rest of the application code. This ensures that if only source code changes, Docker reuses the cached pm install layer, speeding up builds significantly.
- **Production Mode:** --omit=dev ensures only production dependencies are installed.

8 Security Practices

The following security best practices were verified: 1. **Least Privilege:** The API container runs with USER node instead of the default root user. 2. **Network Isolation:** The Database and API have no external ports exposed. Only Nginx is accessible. 3. **Git Hygiene:** .gitignore successfully excludes .env files and ode_modules. No secrets were committed to the GitHub repository.

9 Failure-Mode Scenarios

Extensive runtime testing was performed to verify system resilience:

Scenario	Resulting Status Code / Behavior	Explanation
API container is stopped	502 Bad Gateway	Nginx cannot reach the upstream backend.
DB container is stopped	500 Internal Server Error	API is running but fails to execute queries.
Nginx is stopped	Connection Refused	The external entry point is closed.
Invalid Task ID requested	404 Not Found	Handled properly by API logic.
Invalid POST Payload	400 Bad Request	Handled properly by API validation.

10 Testing Evidence

A native Node.js test script (pi/tests/test.js) acts as an automated integration test. - **Automated Test:** It sends a GET /api/tasks request to localhost:8080. - **Success Criteria:** It expects an HTTP 200 OK status and exits with code . If the API fails, it exits with code 1. - **Manual Runtime Tests:** Operations like POST, PUT, DELETE, and failure-mode triggering (stopping containers) were executed manually and successfully verified at runtime.

11 Registry & Versioning

The API Docker image was published to a public registry (Docker Hub). - **Repository:** hishamdiv/task-management-api - **Tags:** 1.0.0 and latest - **Digest:** sha256:799e625d06f9fdacb5a12da1edc08556a029c7c809e3ec2b238005803cba694d - **Verification:** A “Clean Pull Workflow” was executed by removing local images and successfully pulling hishamdiv/task-management-api:v1.0.0 from Docker Hub.

12 Conclusion

The Task Management API project successfully fulfills all requirements of the Cloud Computing and Containerization specification. Through the strategic use of Docker, Nginx, and Node.js, the project demonstrates a robust, portable, and secure architecture capable of handling failures gracefully while ensuring data persistence.